

Identificação de Caminhos Mínimos em Grafos Direcionados

Adriano Araújo Domingos dos Santos¹

¹Instituto de Ciências Exatas e Informática – Pontifícia Universidade
Católica de Minas Gerais (PUC Minas)

R. Cláudio Manoel, 1162 – Savassi - 30.140-100 – Belo Horizonte – MG – Brazil

{adriano.domingos}@sga.pucminas.br

1. Implementação

A solução desenvolvida consiste na implementação do algoritmo de Dijkstra para grafos direcionados e ponderados, garantindo a seleção do caminho mínimo que também contenha o menor número de arestas. A aplicação foi desenvolvida em Java sem nenhuma dependência externa, salvo para os testes unitários que não são necessários para sua execução.

1.1. Dijkstra

A busca pelo caminho mínimo foi desenvolvida utilizando o algoritmo de Dijkstra em sua forma clássica, utilizando o critério de desempate baseado na quantidade de arestas no caminho. A exploração do grafo é guiada por uma Fila de Prioridade (`PriorityQueue`) que utiliza fundamentalmente um *Min-Heap*, garantindo que o vértice com a menor distância acumulada registrada seja sempre o próximo a ser processado.

Durante a exploração da vizinhança de um vértice u , o relaxamento da aresta direcionada ao vizinho v com peso w ocorre exclusivamente mediante uma condição estrita de melhora de custo. O algoritmo verifica se a nova distância acumulada é estritamente menor que a distância atualmente mapeada para v . Caso verdadeiro, a distância no vetor `distances` é atualizada, o predecessor no vetor `parents` é reescrito e o vértice é readicionado à fila de prioridade.

1.2. Representação *Forward-Star*

A representação *Forward Star* é uma estrutura de dados baseada em vetores contíguos, que serializa a topologia do grafo em três vetores primitivos unidimensionais: `pointers`, `targets` e `weights`. Ela foi escolhida por conta de seu acesso rápido à adjacência dos vértices e excelente uso da localidade de cache.

Na implementação desenvolvida, a construção do grafo foi estritamente desacoplada de sua representação final através do padrão de projeto *Builder*. Durante a fase de construção, as arestas são inseridas em vetores temporários paralelos. Ao finalizar a instanciação, o construtor ordena as arestas por vértice de origem, compacta os destinos no vetores finais `targets` e `weights` de tamanho $|E|$, e preenche o vetor `pointers` de tamanho $|V| + 1$. Cada índice v em `pointers` armazena o índice inicial de seus vizinhos no vetor `targets`, permitindo que a varredura da vizinhança $\Gamma(v)$ ou dos sucessores $\Gamma^+(v)$ de qualquer vértice ou o cálculo do seu grau sejam realizados em tempo $\mathcal{O}(1)$.

1.3. Gerador de Grafos Sintéticos

Para viabilizar a análise de desempenho em cenários massivos e controlados, foi desenvolvida uma ferramenta própria de geração de grafos sintéticos. O gerador é capaz de instanciar topologias direcionadas e não direcionadas, parametrizadas pela quantidade exata de vértices ($|V|$), arestas ($|E|$) e requisitos de conectividade.

A fim de satisfazer a conectividade exigida (Simplesmente ou Fortemente Conexo), a estratégia de construção foi dividida em três etapas fundamentais:

1. **Garantia de Conectividade Base:** Inicialmente, um vetor contendo todos os identificadores de vértices é embaralhado de forma aleatória utilizando o algoritmo de *Fisher-Yates*. Em seguida, itera-se sobre esse vetor conectando o vértice i ao $i + 1$, formando um caminho Hamiltoniano que garante a alcançabilidade de todos os vértices. Para grafos fortemente conexos, uma aresta de fechamento (do último vértice para o primeiro) é adicionada, transformando o caminho em um ciclo.
2. **Preenchimento de Densidade:** Para atingir a cardinalidade $|E|$ desejada, o algoritmo entra em um laço de amostragem uniforme, sorteando pares de vértices (u, v) aleatoriamente. Para assegurar que o grafo resultante seja um grafo simples (sem arestas múltiplas), as conexões geradas são validadas contra um conjunto de dispersão (`HashSet`), que armazena as arestas para buscas em tempo $\mathcal{O}(1)$ utilizando empacotamento de bits para diminuir o custo de armazenamento.
3. **Ponderação:** Após a definição da topologia, a matriz de pesos é populada. Cada aresta recebe um valor inteiro sorteado mediante uma distribuição uniforme dentro do limite parametrizado pelo usuário.

Essa abordagem garante que as instâncias geradas para o experimento sejam conexas, livres de laços e arestas paralelas.

2. Metodologia

Para avaliar a eficácia e a eficiência da implementação, foi desenvolvido um script automatizado de *benchmarking*. O experimento foi desenhado para testar os limites da estrutura e do algoritmo utilizando concorrência (*threads*) para acelerar a execução das múltiplas tentativas.

2.1. Design do Experimento e Parâmetros

Para garantir a significância estatística e mitigar anomalias causadas por processos de *background* do sistema operacional ou pelo *Garbage Collector* da JVM, cada cenário único foi executado em 10 repetições independentes. Os parâmetros estabelecidos foram:

- **Topologia do Grafo:** Foram avaliados grafos sintéticos direcionados sob duas conectividades: *Conexo* (garante conectividade básica) e *Fortemente Conexo* (garante caminhos de ida e volta entre qualquer par de vértices, gerando mais ciclos).
- **Tamanho do Grafo:** Foram gerados grafos em quatro escalas crescentes de magnitude. Os pares de vértices ($|V|$) e arestas ($|E|$) utilizados foram: (1.000, 100.000), (10.000, 1.000.000), (100.000, 10.000.000) e (1.000.000, 100.000.000).

A combinação ortogonal de todas essas variáveis (2 tipos $\times$ 4 tamanhos) executada 10 vezes resultou em um espaço amostral de 80 execuções documentadas.

2.2. Fluxo de Execução e Coleta de Dados

Para cada iteração de teste, o fluxo seguiu um controle rigoroso para isolar os tempos:

- 1. **Geração:** Instanciação do grafo sintético definindo aleatoriamente os pesos entre 1 e 100.
- 2. **Construção da Estrutura:** Transcrição das arestas geradas para a estrutura de dados utilizando a representação *Forward Star*.
- 3. **Busca dos Distâncias:** Execução isolada do algoritmo de Dijkstra, encontrando o caminho mínimo para todos os vértices dado o vértice inicial aleatório.
- 4. **Registro:** Coleta e exportação das métricas de desempenho.

2.3. Métricas de Avaliação

As métricas coletadas incluíram o tempo de geração do grafo, o tempo de processamento efetivo do algoritmo de Dijkstra, tamanho do caminho encontrado e o tipo do grafo.

3. Resultados

Tabela 1. Desempenho médio do Dijkstra em grafos por tipo de conectividade.

V	E	Conectividade	Qtd. Arestas	Custo Médio	Tempo (ms)
1.000	100.000	Simplesmente Conexo	5.00	10.00	7.40
10.000	1.000.000	Simplesmente Conexo	6.80	13.60	42.50
100.000	10.000.000	Simplesmente Conexo	7.60	17.50	471.70
1.000.000	100.000.000	Simplesmente Conexo	9.70	19.50	6052.00
1.000	100.000	Fortemente Conexo	5.30	10.50	3.60
10.000	1.000.000	Fortemente Conexo	7.40	12.90	43.50
100.000	10.000.000	Fortemente Conexo	8.30	17.20	452.00
1.000.000	100.000.000	Fortemente Conexo	9.00	20.60	5649.60

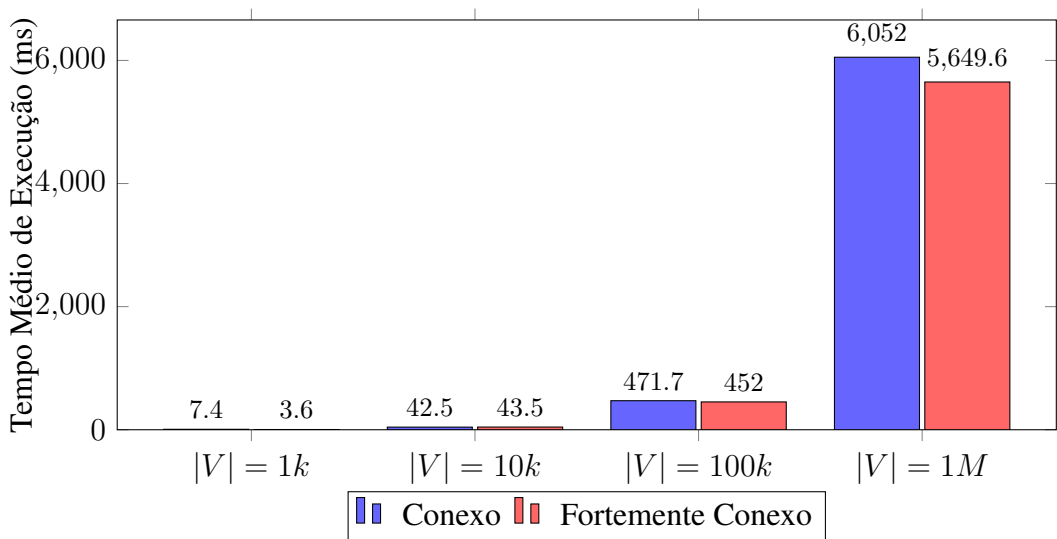


Figura 1. Comparação do impacto da conectividade no tempo de execução do Dijkstra.

Os resultados consolidados na Tabela 1 atestam a alta eficiência e escalabilidade da solução proposta. A execução do algoritmo para instâncias de tamanho reduzido

($|V| = 1.000$ e $|E| = 100.000$) ocorreu em tempo da ordem de 7 milissegundos. De forma expressiva, ao escalar a entrada em 1.000 vezes, submetendo o algoritmo a um cenário massivo de 1 milhão de vértices e 100 milhões de arestas, o tempo de execução manteve-se na casa dos 5,8 segundos.

Ao analisar o impacto da conectividade (visualizado na Figura 1), observa-se que a densidade de ciclos em grafos Fortemente Conexos não gerou um sobrecusto computacional significativo em relação aos Simplesmente Conexos. Para a instância de maior magnitude, a diferença de tempo foi de apenas ≈ 25 milissegundos (5.838,60 ms contra 5.863,40 ms), evidenciando que as rotinas de relaxamento e verificação de desempate possuem custo constante robusto o suficiente para lidar com diferentes distribuições de topologia.

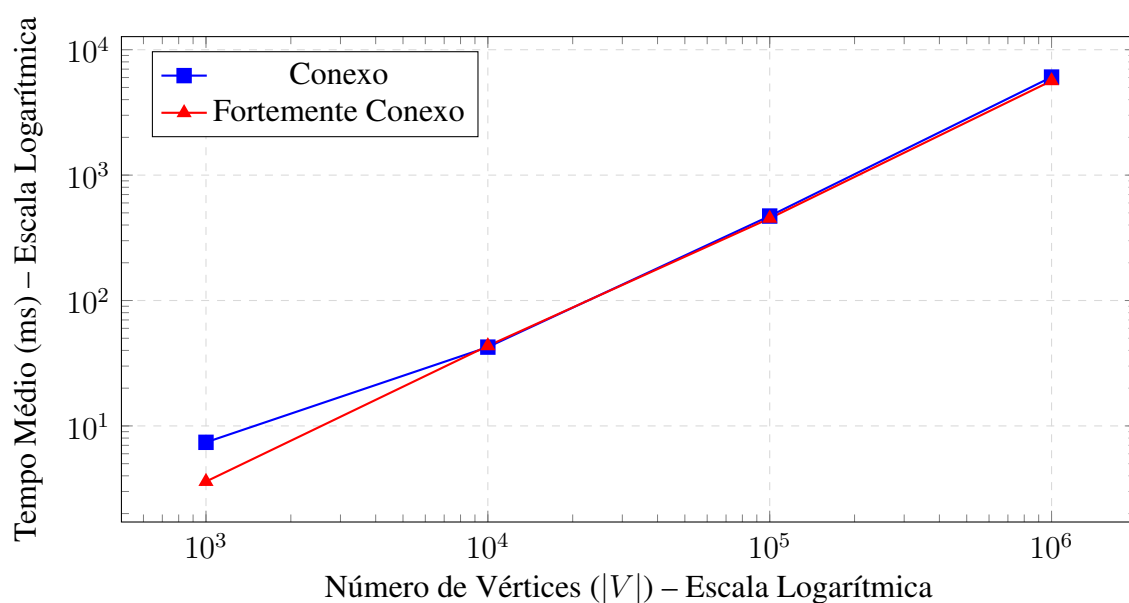


Figura 2. Crescimento logarítmico do tempo de execução do Dijkstra em relação ao tamanho da entrada.

Em relação a quantidade de arestas no caminho mínimo, os resultados indicam que houveram grandes saltos, de modo que mesmo no grafo de 100 milhões de arestas, a média do tamanho do caminho ficou restrita a aproximadamente 9 a 10 saltos. Isso pode ser explicado pelo fato do gerador de grafos criar um caminho inicial contendo todos os vértices.

4. Conclusão

Os resultados encontrados demonstraram a robustez da implementação do algoritmo clássico de Dijkstra quando aliado a estruturas de dados eficientes e baixo nível de abstração geométrica na memória. A utilização combinada da representação *Forward Star* confirmou-se eficaz para viabilizar processamentos em larga escala, permitindo que instâncias com dezenas de milhões de arestas fossem varridas e resolvidas na casa dos segundos, refletindo um alto aproveitamento da localidade de cache.